

Trazado de rayos acelerado con BVH: comparación de heurísticas de construcción sobre tres *backends* (CPU, GPU y Embree)

CS3014 – Estructura de Datos Avanzados (2026-1)

Kalos Lazo Lenin Chávez Marcelo Chinchá

Resumen

El trazado de rayos por fuerza bruta prueba cada rayo contra todos los triángulos ($O(P \cdot N)$ por frame), lo que es inviable en tiempo real. Una jerarquía de volúmenes envolventes (BVH) reduce el costo a $\sim O(\log N)$ por rayo. Implementamos un trazador de rayos con BVH y estudiamos dos preguntas: (i) ¿qué *heurística de construcción* conviene para geometría estática frente a dinámica?, y (ii) ¿cuánto pesa el *hardware* y la ingeniería del recorrido? Para ello ejecutamos el *mismo* árbol y escena sobre tres *backends* —nuestro BVH en CPU multihilo, un *kernel* propio en GPU (OpenCL) e Intel Embree— y barremos tres heurísticas (SAH, mediana, Morton) en tres tamaños de escena. SAH da el mejor árbol ($4,8\times$ menos nodos por rayo que Morton) y el render más rápido en CPU; la GPU acelera $4\text{--}10\times$ recorriendo el mismo árbol; y el traversal SIMD de Embree se mantiene casi constante aunque el árbol empeore. Y observamos que, salvo en un régimen *build-bound*, reconstruir el árbol cada frame no basta para que Morton compense su peor recorrido.

1. Motivación y problema

El trazado de rayos resuelve la visibilidad simulando el camino de la luz: por cada píxel se lanza al menos un rayo y, con sombras, reflejos e iluminación indirecta, varios más. Para P píxeles y N triángulos, la fuerza bruta cuesta $O(P \cdot N)$ por frame; a 1280×720 con 2×10^5 triángulos son $\sim 10^{11}$ pruebas por frame. El problema, entonces, es *estructural*: hace falta una estructura de datos que descarte grupos enteros de triángulos con pruebas baratas. Un segundo eje del problema, poco tratado en cursos, es la **frecuencia de cambio de la geometría**: una escena real es *mixta* —parte fija (edificios, terreno) y parte en movimiento (personajes, *mesh* animado)—, y cada parte impone un compromiso distinto entre costo de construcción y calidad del árbol.

2. Trabajo relacionado

El BVH domina la aceleración de trazado de rayos por su memoria $O(N)$ y robustez. La *heurística de área de superficie* (SAH) fue introducida por MacDonald y Booth [1] y hecha práctica mediante *binning* por Wald [2]. En el otro extremo, las construcciones lineales tipo LBVH ordenan los primitivos por código de Morton para un *build* muy rápido [3, 4]. La intersección rayo-triángulo usa el método de Möller-Trumbore [5]. Como referencia industrial usamos Intel Embree [6], cuyo rendimiento proviene de *kernels* SIMD sobre árboles multi-vía (MBVH). Nuestra contribución no es un algoritmo nuevo, sino un **estudio experimental controlado** que aísla el efecto de la heurística del efecto del hardware, corriendo la misma estructura en tres *backends*.

3. Solución propuesta

BVH y heurísticas. Construimos el árbol *top-down*: en cada nodo se elige el eje de mayor extensión de centroides y un plano de corte que *depende de la heurística*. SAH minimiza el costo esperado de un corte,

$$C_{\text{split}} = 1 + \frac{A_L n_L + A_R n_R}{A_{\text{padre}}},$$

evaluado con 12 *bins* por eje ($O(N \log N)$, casi óptimo). *Median* corta por la mediana de centroides; *Morton* ordena por código Z y parte el índice medio (*build* mínimo, árbol peor). El recorrido es iterativo con pila fija: prueba rayo-AABB por *slabs* y poda todo subárbol cuya caja no se golpea o queda más lejos que el mejor impacto.

Dos BVH: estático + dinámico. Mantenemos dos árboles: uno *estático* para la geometría fija, construido una vez con SAH (su *build* se amortiza), y uno *dinámico* reconstruido cada frame para los objetos en movimiento. Cada rayo consulta ambos y conserva el impacto más cercano; es, en esencia, una TLAS de dos instancias hecha a mano en la que la heurística se elige según la frecuencia de cambio.

Tres backends sobre la misma escena. Comparamos tres motores de trazado:

Backend	Ejecución	Arbol
CPU propio	CPU	BVH propio
GPU OpenCL	GPU	BVH (aplanado) propio
Embree	CPU	Embree optimized BVH

CPU y GPU son el *mismo* raytracer nuestro sobre el mismo árbol —solo cambia el hardware—, mientras que Embree es la referencia industrial externa. Los tres son trazado *por software* (sin núcleos RT dedicados) y comparten el sombreado, de modo que la comparación aísla la estructura y su recorrido. En la GPU el algoritmo cambia: como no admite punteros de host ni recursión, **aplanamos** el árbol a arrays paralelos (`node_bounds`, `node_links` = (`izq,der,inicio,cuenta`), `tris`) y el *kernel* lo recorre con una **pila explícita**.

4. Evaluación

Montaje. CPU AMD Ryzen 7 5700X3D (8 núcleos, 16 hilos), GPU NVIDIA GTX 1650 SUPER (OpenCL), Embree 4; *build* en modo *release* (`-O3`). Escena: campo de esferas con densidad variable —Small/Medium/Large = 12 812/51 212/204 812 triángulos. Todo es reproducible con la bandera `--bench`, que escribe `docs/benchmark.csv` (matriz de 3 *backends* $\times$ 3 heurísticas $\times$ 3 tamaños). Medimos: *render/frame* (frame completo, la métrica comparable entre los tres backends), *nodos/rayo* (calidad del árbol, independiente del hardware), *build* (construcción del árbol) y *traversal* (intersección de rayos primarios en un hilo, que aísla el motor de recorrido para el contraste con Embree).

Cuadro 1: Escena *Large* (204 812 triángulos). Los tres backends rinden un frame completo (métrica comparable); *nodos/rayo* es una propiedad del árbol, que CPU y GPU comparten por recorrer el mismo. CPU y GPU se degradan con la heurística (usan nuestro árbol); Embree no (usa el suyo).

Backend	Métrica	SAH	Median	Morton
CPU propio	render/frame (ms)	18.2	33.5	106.4
GPU OpenCL	render/frame (ms)	3.2	4.5	29.0
Embree	render/frame (ms)	4.6	4.7	7.9
Árbol (propio)	nodos/rayo	138	265	660
CPU propio	build (ms)	84.4	73.8	56.8

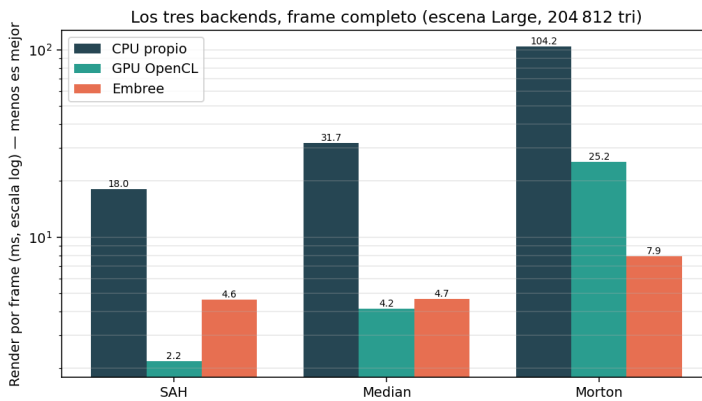


Figura 1: Los tres backends, frame completo (Large, escala log). CPU y GPU usan *nuestro* árbol y se disparan con Morton; Embree usa el suyo y apenas cambia. Con buen árbol la GPU gana; con árbol malo, Embree es el más robusto.

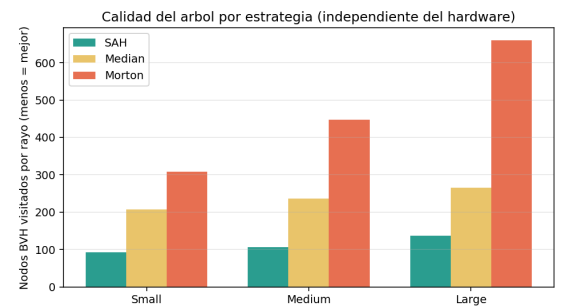


Figura 2: Calidad del árbol (nodos/rayo, menos es mejor): SAH domina en toda densidad. La GPU hereda esta calidad al recorrer el mismo árbol.

Análisis. (i) *Heurística*: SAH visita $4.8\times$ menos nodos por rayo que Morton (138 vs. 660), lo que se traduce en un render en CPU $\sim 5.8\times$ más rápido; Morton solo gana en tiempo de *build*. Como el árbol estático se construye una vez y se recorre millones de veces, SAH es el ganador global para geometría fija. (ii) *Hardware*: la GPU recorre el *mismo* árbol 4–10 $\times$ más rápido que la CPU; el paralelismo masivo por píxel compensa el costo del *flatten* y de la pila explícita, aun sin hardware de trazado dedicado. (iii) *Ingeniería del recorrido*: el traversal de Embree se mantiene en ~ 11 –16 ms aunque el árbol empeore de HIGH a LOW, mientras que nuestro traversal escalar se dispara de 31 a 116 ms; la brecha con Embree está en el *kernel* SIMD/MBVH, no en la calidad del árbol.

¿Y la geometría dinámica? Repetimos el barrido reconstruyendo el árbol *cada frame* sobre una multitud escalable (Fig. 3). El *build* de Morton es el más barato, pero en esta escena sigue siendo pequeño frente al traversal, así que SAH gana el costo por frame. La lectura no es que “lo dinámico favorezca a SAH”, sino que **esta escena no es lo bastante exigente** para que el *build* domine: Morton solo convendría en un régimen *build-bound* (mucho más geometría, muchas reconstrucciones por frame, o muy pocos rayos).

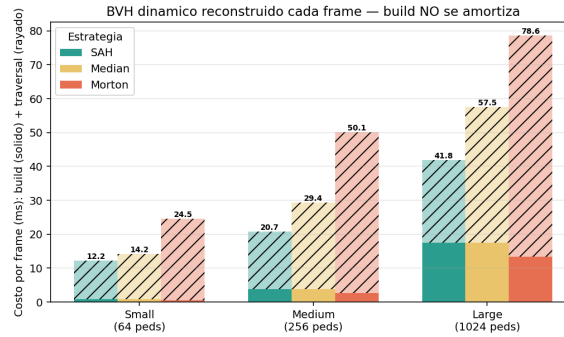


Figura 3: BVH reconstruido cada frame: costo = *build* (sólido) + traversal (rayado). El *build* de Morton es menor, pero su traversal lo hunde y SAH gana el total igualmente.

5. Conclusiones

Un *benchmark* reproducible que, sobre la misma estructura y escena, separa el efecto de la heurística del efecto del hardware. SAH ofrece el mejor árbol (y el mejor render estático); la GPU acelera 4–10× al portar el recorrido; y la ventaja de Embree reside en la ingeniería SIMD de su recorrido, no en la estructura —una distinción central para un curso de estructuras de datos. **Trabajo futuro:** recorrido SIMD y MBVH propios, construcción paralela, *refit* en vez de rebuild para lo dinámico, y comparación con SBVH.

Contribuciones por autor.

- Kalos Lazo: Integración con embree y *benchmark*.
- Lenin Chávez: escena, sombreado y renderer multihilo.
- Marcelo Chinchá: backend OpenCL/GPU, BVH y heurísticas de construcción.

Enlaces.

- Repositorio: <https://github.com/marcelochincha/raytrec>
- Video demo: [Google Drive](#)
- Presentación: [Google Drive](#)

Datos y figuras en el repositorio: `docs/benchmark.csv`, `docs/fig_*.png`, `docs/plot.py`.

Referencias

- [1] J. D. MacDonald y K. S. Booth. *Heuristics for ray tracing using space subdivision*. The Visual Computer, 1990.
- [2] I. Wald. *On fast construction of SAH-based bounding volume hierarchies*. IEEE Symp. on Interactive Ray Tracing, 2007.
- [3] C. Lauterbach et al. *Fast BVH construction on GPUs*. Computer Graphics Forum (Eurographics), 2009.
- [4] T. Karras. *Maximizing parallelism in the construction of BVHs, octrees, and k-d trees*. High-Performance Graphics, 2012.
- [5] T. Möller y B. Trumbore. *Fast, minimum storage ray/triangle intersection*. Journal of Graphics Tools, 1997.
- [6] I. Wald, S. Woop, C. Benthin, G. S. Johnson y M. Ernst. *Embree: a kernel framework for efficient CPU ray tracing*. ACM TOG (SIGGRAPH), 2014.